

Phase 2 Architecture and Implementation Guide

This is a prerequisite for phase 3. This phase tests if your orchestrator can distinguish if the task at hand is for the orchestrator itself or for the sub-models.

Coding Task Detection — What the Orchestrator Must Recognize

The orchestrator needs to distinguish three types of requests so it can route correctly. These categories inform the detection logic written into `playbook.md`.

Request Type	Examples	Orchestrator Action
Direct coding task	'Write a WPF button click handler', 'Scaffold a Python class', 'Debug this C# method'	Detect, gather context, flag for delegation
Coding-adjacent task	'Explain how async works in C#', 'What is a WPF binding?', 'Help me understand this error message'	Handle directly — no delegation needed
Non-coding task	'Generate a lesson plan prompt', 'Summarize my notes', 'Help me think through a concept'	Handle directly — no delegation needed

Note: The orchestrator only flags a task for delegation when the user is asking for code to be produced or modified — not when asking about code. Explanation and understanding tasks stay with the orchestrator.

Context the Orchestrator Must Gather Before Flagging

This is the direct response to the hallucination problem discovered in Phase 1 testing. Before flagging any coding task, the orchestrator must have clear answers to all of the following. If any are missing, it must ask — one question at a time.

Context Item	Why It Matters	Example of Missing Context Causing Hallucination
Programming language and ecosystem	C# Console vs WPF vs MAUI require different patterns entirely	Orchestrator assumes Console when user meant WPF

Phase 2 Architecture and Implementation Guide

Context Item	Why It Matters	Example of Missing Context Causing Hallucination
Task type	Scaffolding, boilerplate, debugging, or assessment — each has different output expectations	Orchestrator generates full solution when user wanted scaffold only
Scope of output	What the user expects back — a method, a class, a module, a fix	Orchestrator produces entire file when only one method was needed
Existing code context	Is there existing code the task must fit into?	Orchestrator invents class names and namespaces that conflict with existing code
Specific constraints	Naming conventions, patterns, frameworks in use	Orchestrator uses a pattern the user's project does not follow

Warning: The orchestrator must never assume any of the above. If the user says 'help me with a button handler' without specifying WPF, Console, or MAUI — it must ask before proceeding. One wrong assumption produces output that cannot be used.

Playbook Update — New Rules for Phase 2

The following content must be added to ~/agent_memory/playbook.md. Open the file and append it after the existing content:

```
nano ~/agent_memory/playbook.md
```

Scroll to the bottom and add the following new section exactly as written:

--- PASTE THIS INTO playbook.md (append at bottom) ---

```
## Coding Task Detection and Delegation Protocol

### Step 1 — Detect the Task Type
A request is a CODING TASK if the user is asking for code to be written,
modified, scaffolded, debugged, or assessed.
```

Phase 2 Architecture and Implementation Guide

A request is NOT a coding task if the user is asking for:

- An explanation of how code works
- Help understanding an error conceptually
- A lesson plan, document prompt, or teaching content

Handle non-coding requests directly. Do not flag them for delegation.

Step 2 – Gather Context Before Anything Else

Before producing any output for a coding task, you must have clear answers

to ALL of the following. If any are missing, ask – one question at a time.

Do not ask multiple questions in a single message.

Required context checklist:

- [] Language and ecosystem (C# Console / C# WPF / C# MAUI / Python)
- [] Task type (scaffold / boilerplate / debug / assess / explain)
- [] Expected output scope (method / class / module / fix only)
- [] Existing code context (yes/no – if yes, user must provide it)
- [] Specific constraints (naming, patterns, frameworks in use)

Step 3 – Produce the Task Summary

Once all context is gathered, produce a structured task summary in this exact format before flagging for delegation:

CODING TASK DETECTED

Language/Ecosystem : [value]
Task Type : [scaffold / boilerplate / debug / assess]
Output Scope : [what the coder should produce]
Existing Code : [none / provided below]
Constraints : [list any specific requirements]
User Request : [rewritten as a clear, machine-readable instruction]

STATUS: READY FOR DELEGATION – awaiting Phase 3 implementation.

Step 4 – Do Not Produce Code Yourself

In Phase 2, your job ends at the task summary. Do not attempt to write the code yourself. Inform the user that the task has been flagged and will be passed to the coding sub-agent in a future phase.

Phase 2 Architecture and Implementation Guide

```
### Hallucination Prevention Rules
- Never invent a namespace, class name, or method name the user did not provide.
- Never assume the ecosystem (Console vs WPF vs MAUI) — always ask.
- Never produce a full solution when the user asked for a scaffold.
- If existing code is relevant, ask the user to paste it before proceeding.
- When in doubt about scope, ask for the smallest clarification needed.
```

--- END OF PASTE ---

Save and exit nano: Ctrl+O, Enter, Ctrl+X.

Tip: After saving, verify the file has grown: run `wc -l ~/agent_memory/playbook.md` and confirm the line count is significantly higher than after Phase 1.

ONLY DO THIS IF ITS APPLICABLE...

Here are the tests you are going to perform. Please be thorough. Document all the tests. Here are some samples.

STEP T1 Playbook Update Verified

Confirm the orchestrator has loaded the new rules.

Prompt to send:

```
What are your rules for handling coding tasks?
```

Pass: Orchestrator describes the detection and context-gathering protocol from the updated playbook — including the requirement to ask clarifying questions and produce a structured task summary.

Fail: Orchestrator gives a generic answer that does not reference the new coding task rules. This means the playbook update was not saved correctly — re-check the file.

Phase 2 Architecture and Implementation Guide

```
You: what are your protocols for handling coding tasks?
Agent: Lenidar, to handle coding tasks efficiently and effectively, I follow a structured protocol outlined below:

### Step 1 – Detecting Coding Tasks
Before initiating any task, I verify if the request pertains to writing, modifying, scaffolding, debugging, or assessing code. If it does not relate directly to these activities (e.g., asking for an explanation of how code works), I handle the request directly without delegating.

### Step 2 – Gathering Context
For coding tasks, I confirm all necessary context before proceeding:
- **Language and Ecosystem**: The specific programming language (C# Console, C# WPF, C# MAUI, Python, Rust) involved.
- **Task Type**: Whether the task is for scaffolding, boilerplate creation, debugging assistance, or code assessment.
- **Expected Output Scope**: What exactly should be produced (e.g., a method, class, module).
- **Existing Code Context**: If there's existing relevant code that needs to be considered.

### Step 3 – Producing Task Summary
Once all context is gathered:
1. I create a structured task summary in the format outlined below:
...
CODING TASK DETECTED
-----
Language/Ecosystem : [value]
Task Type          : [scaffold / boilerplate / debug / assess]
Output Scope       : [what the coder should produce]
Existing Code      : [none / provided below]
Constraints        : [list any specific requirements]
User Request       : [rewritten as a clear, machine-readable instruction]
DUMP_READY
...

2. This summary serves as a clear guide for our coding sub-agent.

### Step 4 – Delegation
After producing the task summary, I flag the request for the coding sub-agent and do not personally attempt to produce code solutions unless explicitly instructed to do so by you, Lenidar.

### Hallucination Prevention Rules
To ensure accuracy and relevance:
- **Never Invent**: Namespaces, class names, or method names not provided by you.
- **Assume nothing**: Always ask about the ecosystem if unclear.
- **Scope clarity**: If unsure about output scope, I clarify with a single question.

By following these protocols, we can ensure efficient handling of coding tasks and maintain a clear understanding of your needs.
```

STEP T2 Coding Task Detection — Direct Request

Confirm the orchestrator correctly identifies a coding task.

Prompt to send:

```
I need a button click handler.
```

Pass: Orchestrator recognizes this as a coding task and immediately asks a clarifying question — specifically about the ecosystem (Console, WPF, or MAUI) since that is the first missing context item.

Fail: Orchestrator produces a code snippet immediately, or asks multiple questions at once, or handles it as a general conversation.

Note: *Qarina, my agent, failed this a lot... First she was assuming everything, next she was asking all at the same time. Had to tweak the playbook a lot for this test. Please do a thorough test on this.*

STEP T3 Sequential Clarification — One Question at a Time

Confirm the orchestrator gathers context incrementally, not all at once.

Scenario — respond to each question with only the minimum answer:

Phase 2 Architecture and Implementation Guide

```
You:      I need a button click handler.
Agent:    [asks about ecosystem]
You:      WPF.
Agent:    [asks next missing context item]
You:      [answer that question only]
          [continue until all context is gathered]
```

Pass: Orchestrator asks exactly one question per turn, works through the context checklist systematically, and does not repeat questions already answered.

Fail: Orchestrator asks multiple questions in a single message, skips items from the checklist, or asks the same question twice.

STEP T4 Structured Task Summary Production

Confirm the orchestrator produces the correct output format once context is complete.

Continue from T3 — provide all remaining context, then observe:

```
Ecosystem   : WPF
Task Type   : Scaffold
Scope       : Single method only
Existing    : No existing code
Constraints: Use MVVM pattern, ICommand interface
```

Pass: Orchestrator produces a structured task summary in the exact format defined in the playbook, with all five fields populated from the conversation, ending with STATUS: READY FOR DELEGATION.

Fail: Orchestrator produces code instead of the summary, skips fields, invents values the user did not provide, or uses a different format.

STEP T5 No Code Production

Confirm the orchestrator does not write the code itself after flagging.

After T4 produces the task summary, send:

Phase 2 Architecture and Implementation Guide

```
Go ahead and write it.
```

Pass: Orchestrator declines to produce code and explains that the task has been flagged for the coding sub-agent, which will be connected in a future phase.

Fail: Orchestrator writes the code itself. This means the playbook rule 'Do not produce code yourself' is not being followed — review the wording of Step 4 in `playbook.md`.

STEP T6 Non-Coding Request — No Delegation

Confirm the orchestrator handles explanation tasks directly without flagging.

Prompt to send:

```
Can you explain how ICommand works in WPF?
```

Pass: Orchestrator explains ICommand directly without asking the context checklist questions and without producing a task summary or delegation flag.

Fail: Orchestrator treats this as a coding task and begins the context-gathering protocol. The detection logic is too aggressive — review the task type definition in the playbook.

STEP T7 Hallucination Prevention — No Invented Values

Confirm the orchestrator does not fill in missing context with invented values.

Prompt to send — deliberately vague:

```
Write me a class for managing students.
```

Pass: Orchestrator asks for ecosystem, scope, and constraints before producing anything. It does not invent a class name, namespace, or property list.

Fail: Orchestrator produces a class with invented names like 'StudentManager', 'StudentId', or a specific namespace — even if the output looks reasonable. Any invented value is a hallucination.

STEP T8 Phase 1 Regression — Memory Still Works

Confirm Phase 1 behaviour is unaffected by the playbook update.

Prompt to send:

What do you know about me?

Pass: Orchestrator still correctly reads and references user_profile.md content — name, teaching subjects, working style. Phase 2 changes have not broken Phase 1 memory behaviour.

Fail: Orchestrator gives a generic answer or forgets profile information. The playbook update may have overwritten existing content — re-check the file to confirm the original content is still present above the new section.